



Computer Science & IT

Data Structure & Programming



Linked List

Lecture No. 01



By- Abhishek Sir

Recap of Previous Lecture



Topic

Circular queue

Topic

Queue using stack.

Topic

Topic

Topic

Topics to be Covered



Topic

Dynamic Memory allocation

Topic

problem with pointers

Topic

Single linked List basic function

Topic

Time complexity of operation is $O(1)$

Topic



Topic : Single Linked List

```
typedef struct node{  
    int data;  
    struct node *link;  
}  
}node;
```




Topic : Single Linked List

```
typedef struct node{  
    int data;  
    struct node *link;  
}node;
```

Member of structure

Self referencial Node



↓ pointer

Node of linked List

Struct node n₁;
node n₂;



Topic : Single Linked List

```
typedef struct node{  
    int data;  
    struct node *link;  
}node;
```



Member of structure



Topic : Single Linked List



```
typedef struct node{  
    int data;  
    struct node *link;  
}node;
```

$node\ n_1 = \{12, Null\}$

$n_1 \cdot data$

$n_1 \cdot link$



Topic : Single Linked List



```
typedef struct node{  
    int data;  
    struct node *link;  
}node;
```

-> Operator is required to access member through structure Pointer

pointer to structure

$\text{node } n_1 = \{ 12, \text{NULL} \};$

$\text{node } *pt\sigma;$

$pt\sigma = \&n_1;$

$\text{printf} (" \%d", pt\sigma \rightarrow \text{data});$

$\text{printf} (" \%p", pt\sigma \rightarrow \text{link});$



Topic : Dynamic Memory Allocation

```
#include <stdlib.h>
```

```
int a1;
```

```
int main() {
```

```
int a2;
```

```
int *ptr = malloc(sizeof(int));
```

```
}
```

a₁

a₂

ptr

Data Segment

Compile-time
allocation

Runtime
stack

Heap

Run-time
allocate



Topic : Dynamic Memory Allocation

Dynamic Memory allocation is run-time allocation

Static Memory allocation is compile time allocation



Topic : Dynamic Memory Allocation

← type cast void pointer to int pointer

```
int *ptr = (int*) malloc (sizeof (int));
```



malloc return time void pointer



Topic : Dynamic Memory Allocation

Is it mandatory to typecast malloc function??

No : compiler does the type casting

```
int *ptr = malloc(sizeof(int));
```

No Error



Topic : Dynamic Memory Allocation

`malloc/calloc` : Memory allocation done by user

`Calloc` function: Contiguous Allocation,

2 parameters, automatic value assigned to 0

```
int *ptr = calloc(sizeof(int), 5);
```

ptr 1000





Topic : Dynamic Memory Allocation

Deallocation: User will deallocation of Memory

```
int *ptr = malloc(sizeof(int));
```

```
*ptr = 100;
```

```
.....  
free(ptr); // free(Address)
```




Topic : Dynamic Memory Allocation

if user Dynamically allocate memory and does not deallocate
then it leads to **Memory Leak** (Not an Error)

Memory Leak

```
int *ptr = malloc(sizeof(int));
```

```
*ptr = 100;
```

ptr **1000**

100
1000

```
ptr = malloc(sizeof(int);
```

```
*ptr = 200;
```

ptr **2000**

200
2000



Topic : Dynamic Memory Allocation

```
int *ptr = malloc(sizeof(int));
```

```
*ptr = 100;
```

```
...
```

```
free(ptr);
```

```
printf("%d", *ptr);
```

Dangling pointer

pointing to a location
that does not exist

ptr contains

Address that does not
exist



Topic : Dynamic Memory Allocation

Null pointer dereference

getnode function



```
node* getnode(int x) {
```

```
node* p = malloc(sizeof(node));
```

```
p->data = x;
```

```
p->link = NULL;
```

```
return p;
```

```
}
```

getnode function
creates a node

Dynamically (Malloc/calloc)

and returns its address



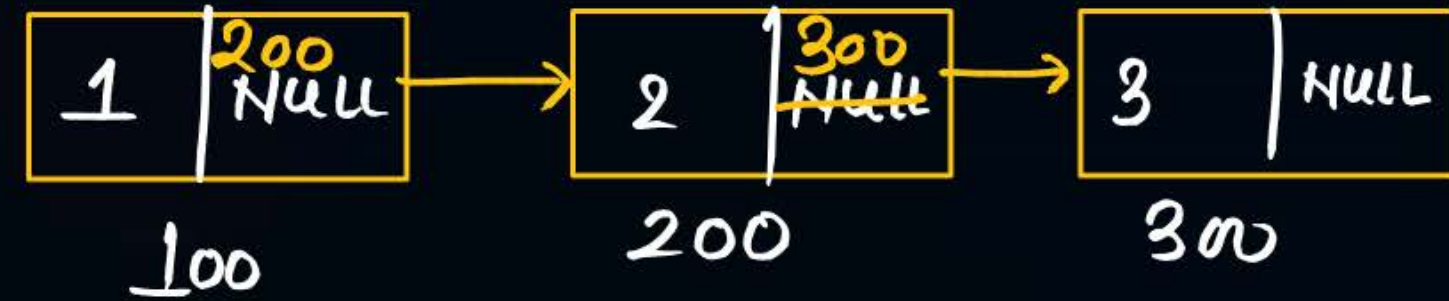
getnode function

```
node* getnode(int x){  
    node *p;  
    p = malloc(sizeof(node));  
    p->data = x;  
    p->link = NULL;  
    return p;  
}
```

Build123() function



```
node* Build123() {  
    node* temp1 = getnode(1);  
    node* temp2 = getnode(2);  
    node* temp3 = getnode(3);  
    temp1 → link = temp2;  
    temp2 → link = temp3;  
    return temp1;  
}
```



temp₁ = 100 temp₂ = 200 temp₃ = 300

2000

Build123() function

```
node * bulid123(){
    node *temp, *temp1, *temp2;
    temp = getnode(1);
    temp1 = getnode(2);
    temp2 = getnode(2);
    temp->link = temp1;
    temp1->link =temp2;
    return temp;
}
```

```
node* HP;
int main(){
    HP = bulid123();
}
```

Head pointer is global variable

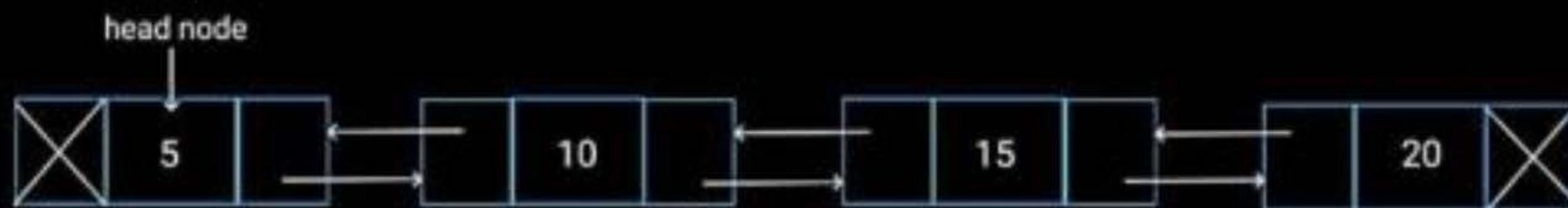


Topic : Linked List

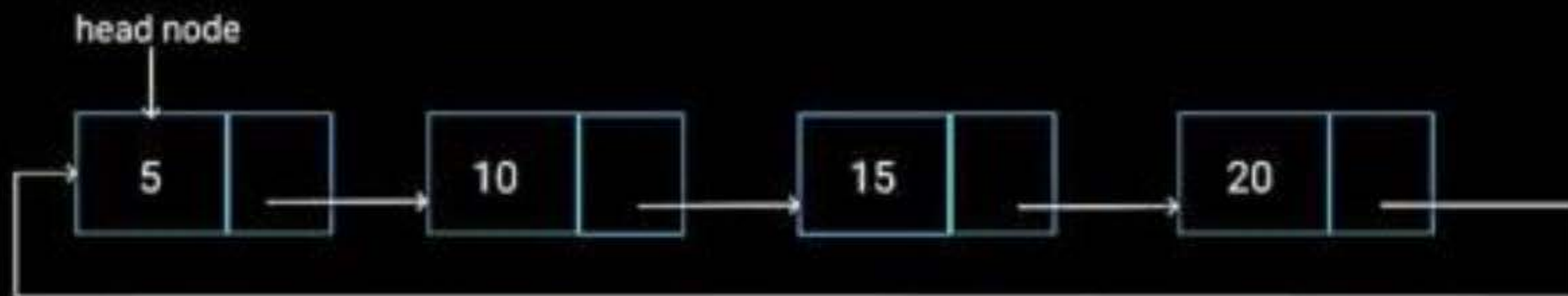
Singly Linked List



Doubly Linked List



Circular Linked List



Count Number of nodes



Head Pointer



pointer

Node

Head pointer contains address of - first Node

All operation in Linked list done by passing Address of first Node (Head pointer)



Traversal and print

Head Pointer



```
void display(node *s) {
```

```
while (s != NULL) { // while(s)
```

```
printf("%d", s->data);
```

```
s = s->link;
```

```
}
```

```
}
```

```
4000 != NULL  
pf(4)  
s = NULL
```

```
NULL != NULL  
false → out of loop
```

```
1000 != NULL  
pf(1)  
s = 2000
```

```
2000 != NULL  
pf(2)  
s = 3000
```

```
3000 != NULL  
pf(3)  
s = 4000
```




Count Number of nodes

Head Pointer



Head pointer contains address of -first Node

All operation in Linked list done by passing Address of first Node (Head pointer)

Count Number of nodes



Head Pointer

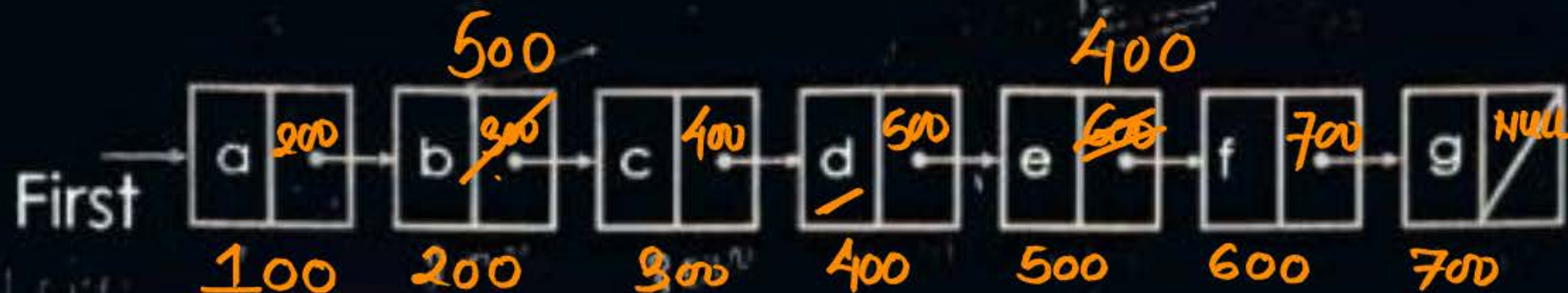


```
int count(node*s) {  
    int counter = 0;  
    while(s != Null) {  
        counter++;  
        s = s->next;  
    }  
    return counter;  
}
```




Topic : Single Linked List

01.



What would be the output after the following sequence of steps?

```
struct node *p;
```

```
p = first → link → link → link;
```

```
p → link → link = p;
```

```
first → link → link = p → link;
```

```
printf ("%c", first → link → link → link → data);
```

(a) a

(b) b

(c) c

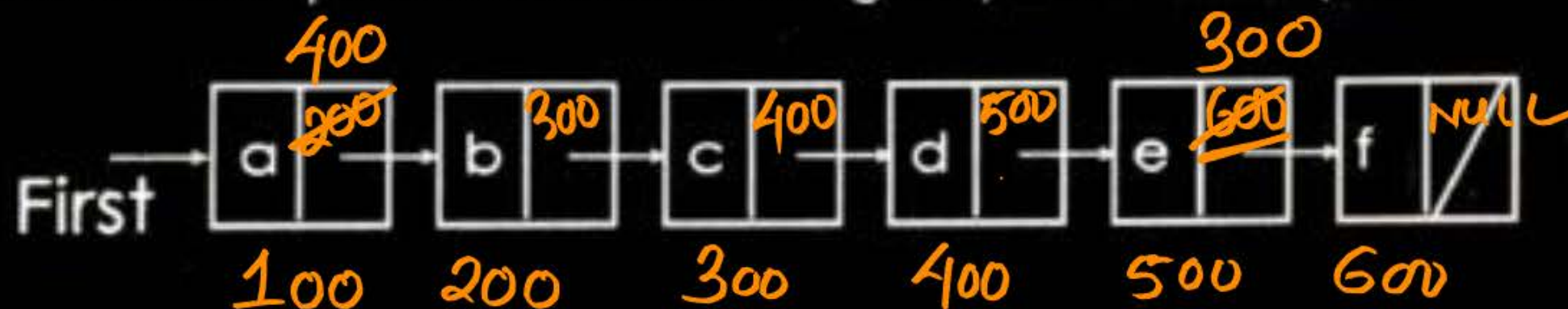
(d) d

p 400



Topic : Single Linked List

02. What is the output after the following sequence of steps?



```
struct node *P;
```

(i) $P = \text{first} \rightarrow \text{link} \rightarrow \text{link};$

(ii) $P \rightarrow \text{link} \rightarrow \text{link} \rightarrow \text{link} = P;$

(iii) $\text{first} \rightarrow \text{link} = P \rightarrow \text{link};$

(iv) `printf ("%c", first → link → link → link → link → data);`

(a) a

(b) b

(c) c

(d) d





If we pass **Input** : head: 1->2->1->2->1, key = 1 to the given function

^{100 200 300 400 500}
int count(struct Node* head, int key) {

int ans = 0;

if (head == NULL) {

return 0;

}

ans = count(head->next, key);

if (head->data == key) {

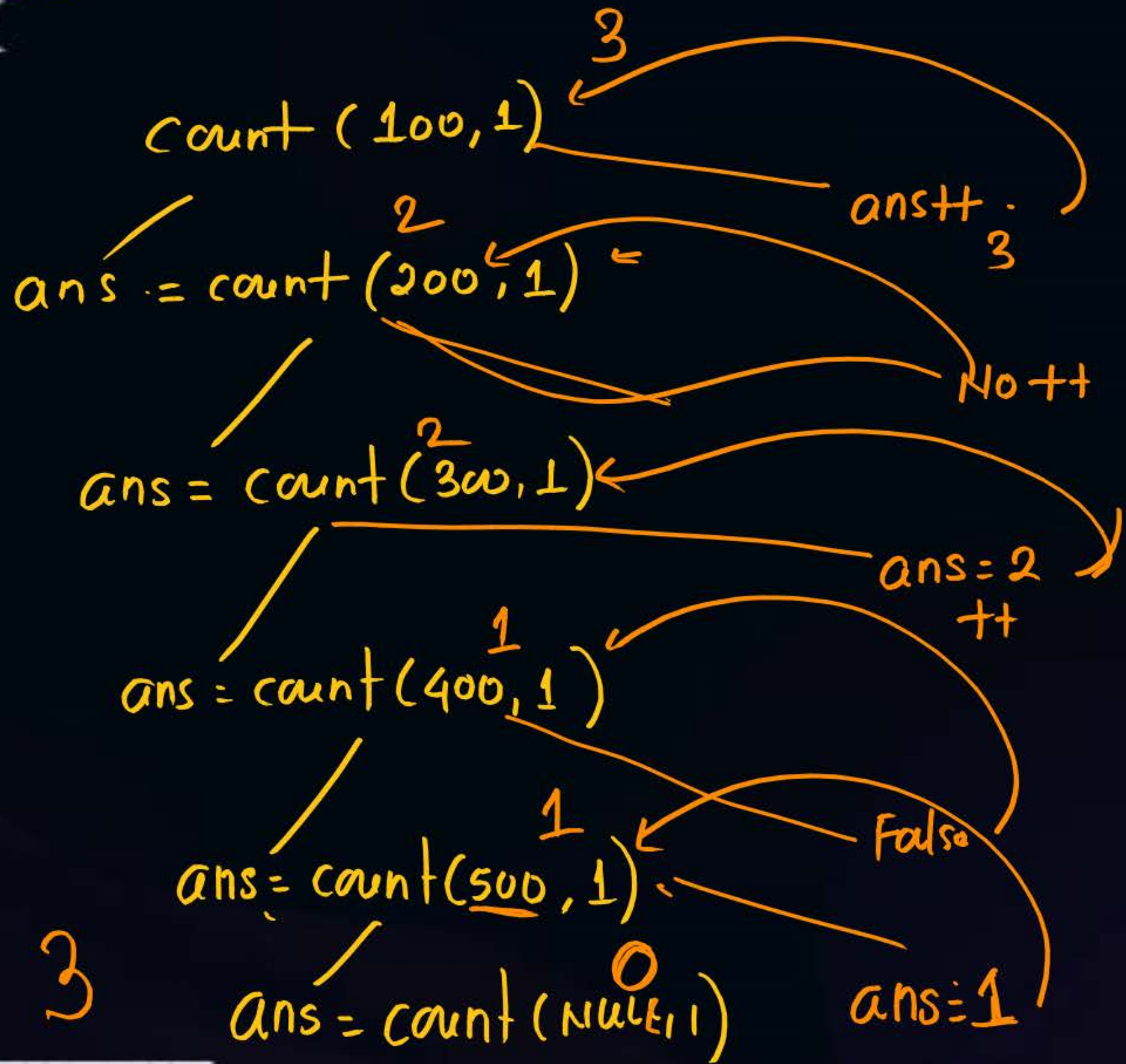
ans++;

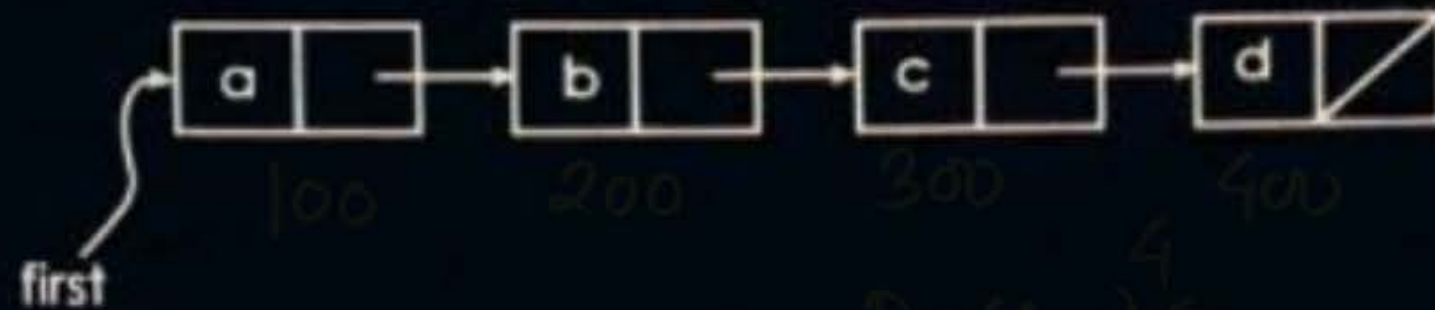
}

return ans;

}

The value return by the program is _____



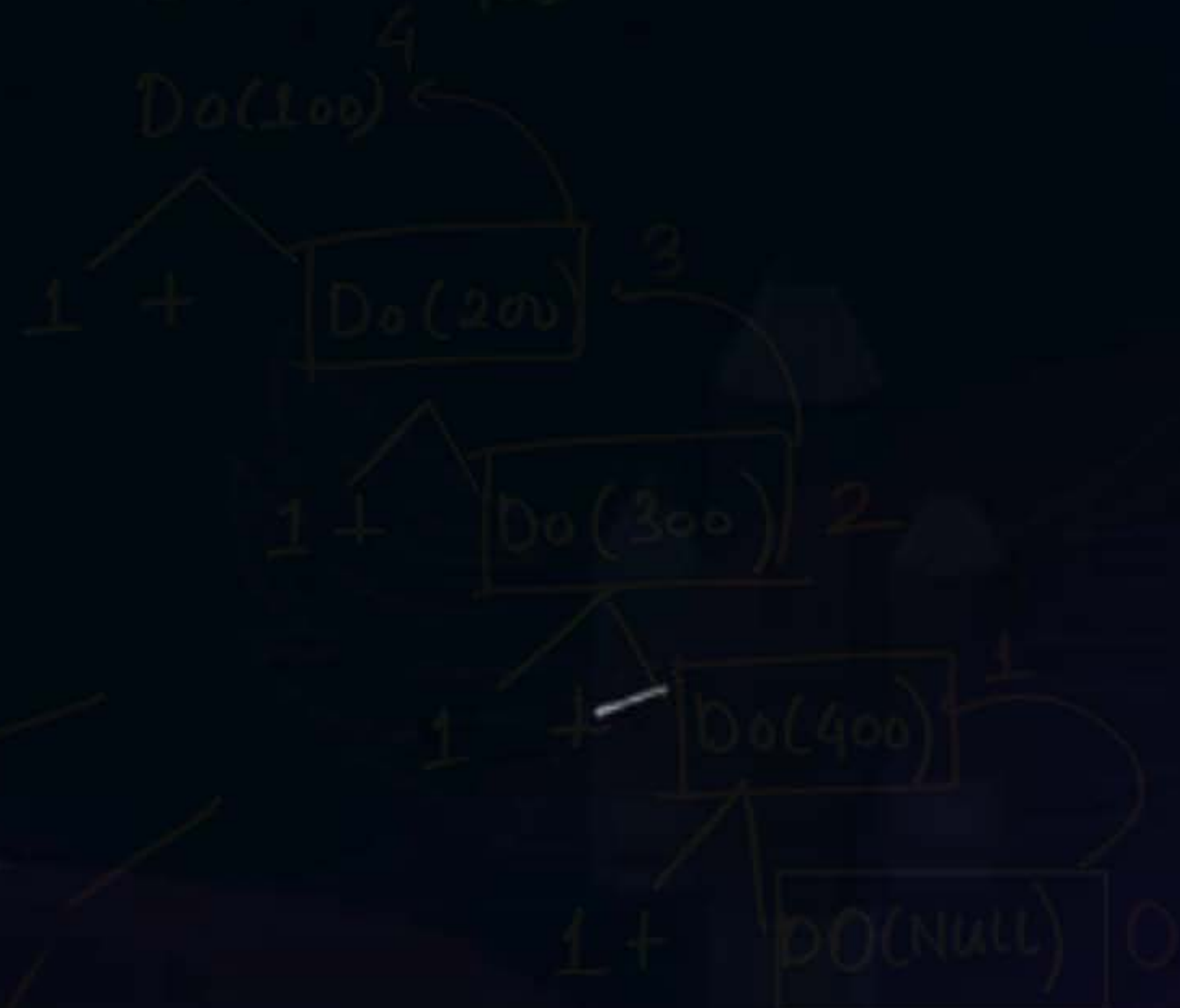


```

int Do (struct node *p)
{
    if (p)
        return 1 + Do (p → link);
    else
        return 0;
}

void main ( )
{
    printf ("%d", Do (first));
}

```



Consider the function

```
node *hp;
```

```
void bar(node *s, int x){
```

```
    node *p =getnode(x);
```

```
        p->next = s;
```

```
        hp = p;
```

```
}
```

```
int main(){
```

```
    for(int i = 34; i<=41; i+=2)
```

```
        bar(hp, i);
```

```
    display(hp);
```

```
    return 0;
```

Assuming HP is global variable and holding the value NULL . What will be the content of the list

A.34 → 36 → 38 → 40 → NULL

B.40 → 38 → 36 → 34 → NULL

C.41 → 39 → 37 → 35 → NULL

D.34 → 38 → 36 → 40 → NULL

Consider the function

```
node *hp;
```

```
void bar(node *s, int x){
```

```
    node *p =getnode(x);
```

```
        p->next = s;
```

```
        hp = p;
```

```
}
```

```
int main(){
```

```
    for(int i = 34; i<=41; i+=2)
```

```
        bar(hp, i);
```

```
    display(hp);
```

```
    return 0;
```


Consider the function

```
node *hp;
```

```
void foo(node *s){
```

```
    while(s != NULL){
```

```
        printf("%d ", s->data);
```

```
        if(s->next == NULL) ✓
```

```
            break;
```

```
        s = s->next->next;
```

```
    }
```

```
}
```

1 , 3 , 5 , 7 , 9

printed if

If the list consists of 1->2->3->4->5->6->7->8->9 number of nodes ~~after~~ we pass

head pointer of the list is _____



Topic Question

Consider the function defined below.

PYQ

2003

HW

```
struct item {  
    int data;  
    struct item * next;  
};  
  
int f(struct item *p) {  
    return ((p == NULL) || (p->next == NULL) ||  
            ((p->data <= p->next->data) &&  
             f(p->next)));  
}
```

For a given linked list p , the function f returns 1 if and only if

A the list is empty or has exactly one element

B the elements in the list are sorted in non-decreasing order of data value

C the elements in the list are sorted in non-increasing order of data value

D not all elements in the list have the same data value

THANK - YOU